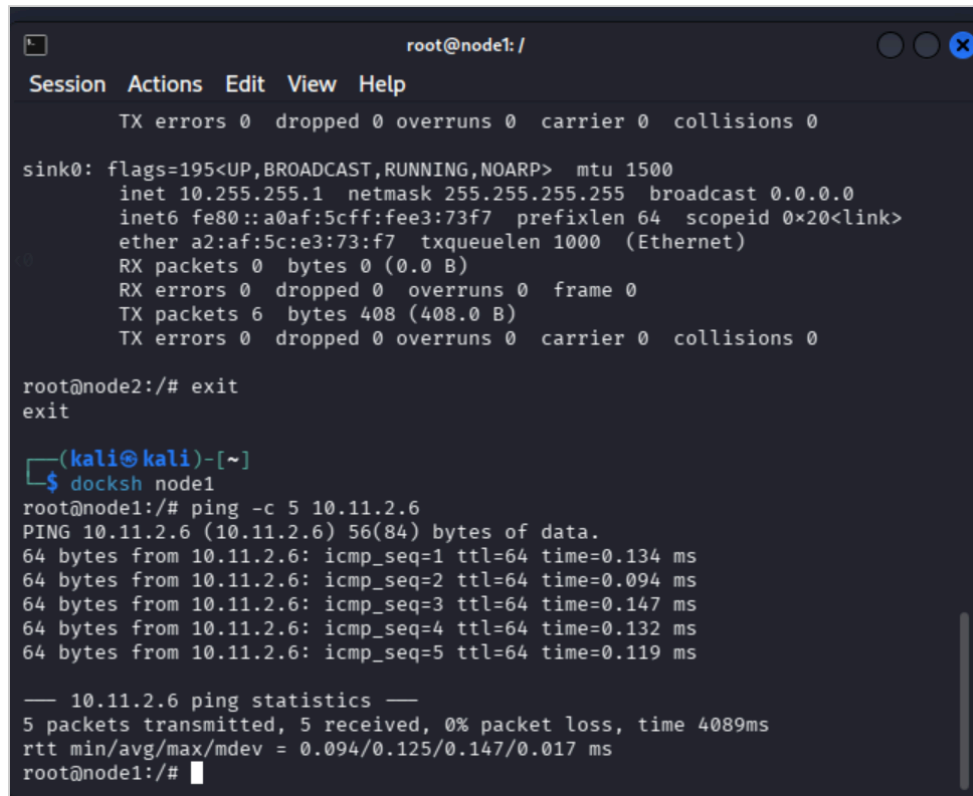


Lab 1: Packet Sniffing and Spoofing

Network Security (50.020) · Name: <Name> · Student ID: <StudentID>

Environment. One Kali VM running three Docker containers on an isolated network 10.11.2.0/24: node1 10.11.2.5 (MAC c2:ea:c3:7e:d6:be), node2 10.11.2.6 (MAC b6:8f:0b:f5:a6:a0), node3 10.11.2.7.

1. Generate Network Traffic



```
root@node1: /
Session Actions Edit View Help
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

sink0: flags=195<UP,BROADCAST,RUNNING,NOARP> mtu 1500
inet 10.255.255.1 netmask 255.255.255.255 broadcast 0.0.0.0
inet6 fe80::a0af:5cff:fee3:73f7 prefixlen 64 scopeid 0<link>
ether a2:af:5c:e3:73:f7 txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 6 bytes 408 (408.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@node2:/# exit
exit

(kali@kali)-[~]
$ docksh node1
root@node1:/# ping -c 5 10.11.2.6
PING 10.11.2.6 (10.11.2.6) 56(84) bytes of data.
64 bytes from 10.11.2.6: icmp_seq=1 ttl=64 time=0.134 ms
64 bytes from 10.11.2.6: icmp_seq=2 ttl=64 time=0.094 ms
64 bytes from 10.11.2.6: icmp_seq=3 ttl=64 time=0.147 ms
64 bytes from 10.11.2.6: icmp_seq=4 ttl=64 time=0.132 ms
64 bytes from 10.11.2.6: icmp_seq=5 ttl=64 time=0.119 ms

— 10.11.2.6 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4089ms
rtt min/avg/max/mdev = 0.094/0.125/0.147/0.017 ms
root@node1:/#
```

Figure 1. `ping -c 5 10.11.2.6` from node1: 5 sent, 5 received, 0% loss, average RTT 0.125 ms.

Q: What does a successful Ping confirm? Does it mean all services on node2 are accessible?

It confirms IP-layer reachability: node2 is up, packets are delivered in both directions, and ICMP is not blocked (5/5 replies, 0% loss). It does **not** mean all services are accessible. Ping tests only ICMP; TCP/UDP services may be stopped, bound to localhost, or blocked by a firewall.

```
Session Actions Edit View Help
(kali@kali)-[~]
$ docksh node1
root@node1:/# telnet 10.11.2.6
Trying 10.11.2.6 ...
Connected to 10.11.2.6.
Escape character is '^]'.

Linux 6.19.14+kali-amd64 (node2) (pts/1)

node2 login: root
Password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.19.14+kali-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
root@node2:~# mkdir test
root@node2:~# ls
test
root@node2:~#
```

Figure 2. Telnet from node1 to node2 as `root`, then `mkdir test` and `ls` on node2.

```
connection closed by foreign host.
root@node1:/# ssh kali@10.11.2.6
kali@10.11.2.6's password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.19.14+kali-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Sep 28 07:31:39 2026 from 10.11.2.5
kali@node2:~$ ls
kali@node2:~$ mkdir test
kali@node2:~$ ls
test
kali@node2:~$
```

Figure 3. SSH from node1 to node2 as `kali`, then `ls`, `mkdir test`, and `ls` on node2.

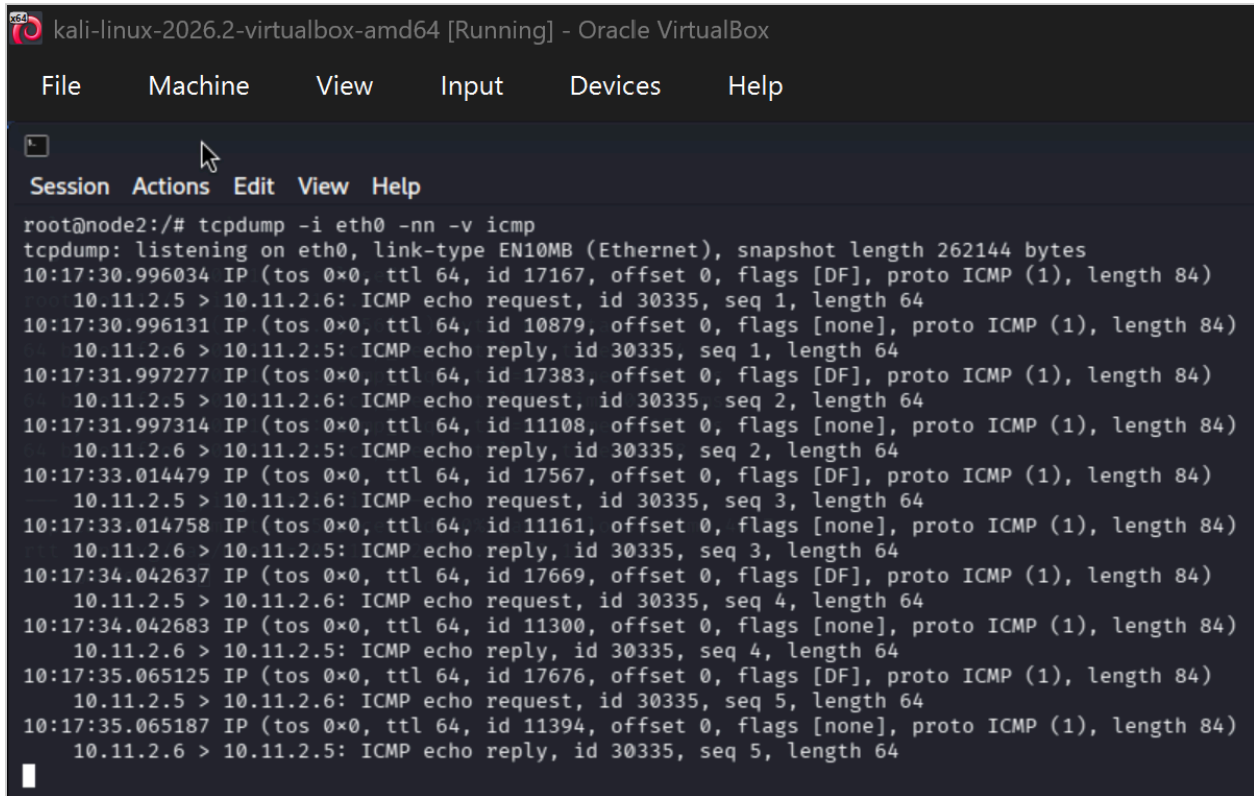
Q: What is the main security weakness of Telnet?

Telnet sends all data, including the username and password, in plaintext. It has no encryption, integrity protection, or server authentication, so anyone who can observe the traffic can steal credentials, read commands, or tamper with or hijack the session. SSH provides all three protections, which makes Telnet unsuitable for modern remote administration.

2. Packet Sniffing and Traffic Analysis

2.1 tcpdump

On node2, `eth0` (10.11.2.6/24) is the interface on the lab network. `lo` is loopback, and `sink0` is a dummy interface for off-subnet routes. tcpdump was run on `eth0` with a filter for each protocol while node1 generated the traffic.



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Session Actions Edit View Help
root@node2:/# tcpdump -i eth0 -nn -v icmp
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10:17:30.996034 IP (tos 0x0, ttl 64, id 17167, offset 0, flags [DF], proto ICMP (1), length 84)
    10.11.2.5 > 10.11.2.6: ICMP echo request, id 30335, seq 1, length 64
10:17:30.996131 IP (tos 0x0, ttl 64, id 10879, offset 0, flags [none], proto ICMP (1), length 84)
    10.11.2.6 > 10.11.2.5: ICMP echo reply, id 30335, seq 1, length 64
10:17:31.997277 IP (tos 0x0, ttl 64, id 17383, offset 0, flags [DF], proto ICMP (1), length 84)
    10.11.2.5 > 10.11.2.6: ICMP echo request, id 30335, seq 2, length 64
10:17:31.997314 IP (tos 0x0, ttl 64, id 11108, offset 0, flags [none], proto ICMP (1), length 84)
    10.11.2.6 > 10.11.2.5: ICMP echo reply, id 30335, seq 2, length 64
10:17:33.014479 IP (tos 0x0, ttl 64, id 17567, offset 0, flags [DF], proto ICMP (1), length 84)
    10.11.2.5 > 10.11.2.6: ICMP echo request, id 30335, seq 3, length 64
10:17:33.014758 IP (tos 0x0, ttl 64, id 11161, offset 0, flags [none], proto ICMP (1), length 84)
    10.11.2.6 > 10.11.2.5: ICMP echo reply, id 30335, seq 3, length 64
10:17:34.042637 IP (tos 0x0, ttl 64, id 17669, offset 0, flags [DF], proto ICMP (1), length 84)
    10.11.2.5 > 10.11.2.6: ICMP echo request, id 30335, seq 4, length 64
10:17:34.042683 IP (tos 0x0, ttl 64, id 11300, offset 0, flags [none], proto ICMP (1), length 84)
    10.11.2.6 > 10.11.2.5: ICMP echo reply, id 30335, seq 4, length 64
10:17:35.065125 IP (tos 0x0, ttl 64, id 17676, offset 0, flags [DF], proto ICMP (1), length 84)
    10.11.2.5 > 10.11.2.6: ICMP echo request, id 30335, seq 5, length 64
10:17:35.065187 IP (tos 0x0, ttl 64, id 11394, offset 0, flags [none], proto ICMP (1), length 84)
    10.11.2.6 > 10.11.2.5: ICMP echo reply, id 30335, seq 5, length 64
```

Figure 4. `tcpdump -i eth0 -nn -v icmp`: only ICMP echo request/reply pairs between node1 and node2.

- **SSH (Figure 6):** TCP port 22; a readable version banner, larger key-exchange packets, then only encrypted payloads.

The "cksum incorrect" messages are caused by checksum offloading: tcpdump sees outgoing packets before the checksum is filled in. They are not transmission errors.

Q: What information can be identified from tcpdump output?

- Timestamp, source and destination IP addresses and ports, and protocol.
- IP header fields (`-v`): TOS, TTL, IP ID, flags (e.g. DF), protocol, and total length.
- TCP flags (SYN, ACK, PSH, FIN), sequence and acknowledgement numbers, window, options, and payload length.
- ICMP type (echo request/reply), identifier, and sequence number.
- Payload contents with `-A` (ASCII) or `-x` (hex and ASCII).

Filter commands (Figures 4–6):

```
tcpdump -i eth0 -nn -v icmp           # Ping only
tcpdump -i eth0 -nn -v -A 'tcp port 23' # Telnet only (ASCII payload)
tcpdump -i eth0 -nn -v 'tcp port 22'    # SSH only
```

Q: Can the Telnet and SSH passwords be seen in the traffic?

Telnet: yes. Each keystroke is sent as a separate cleartext segment, so `-A` prints the password one character per packet (Figure 5). Figure 10 shows the same credentials recovered in Wireshark. **SSH: no.** After the key exchange (Figures 6 and 9), everything, including the password, is encrypted. Only packet sizes and timing are visible.

2.2 Wireshark

Requirement	Display filter	Result (of 212 captured packets)
Packets sent by node2 only	<code>ip.src == 10.11.2.6</code>	94 displayed
Telnet traffic only	<code>telnet (or tcp.port == 23)</code>	44 displayed
SSH traffic only	<code>ssh (or tcp.port == 22)</code>	91 displayed

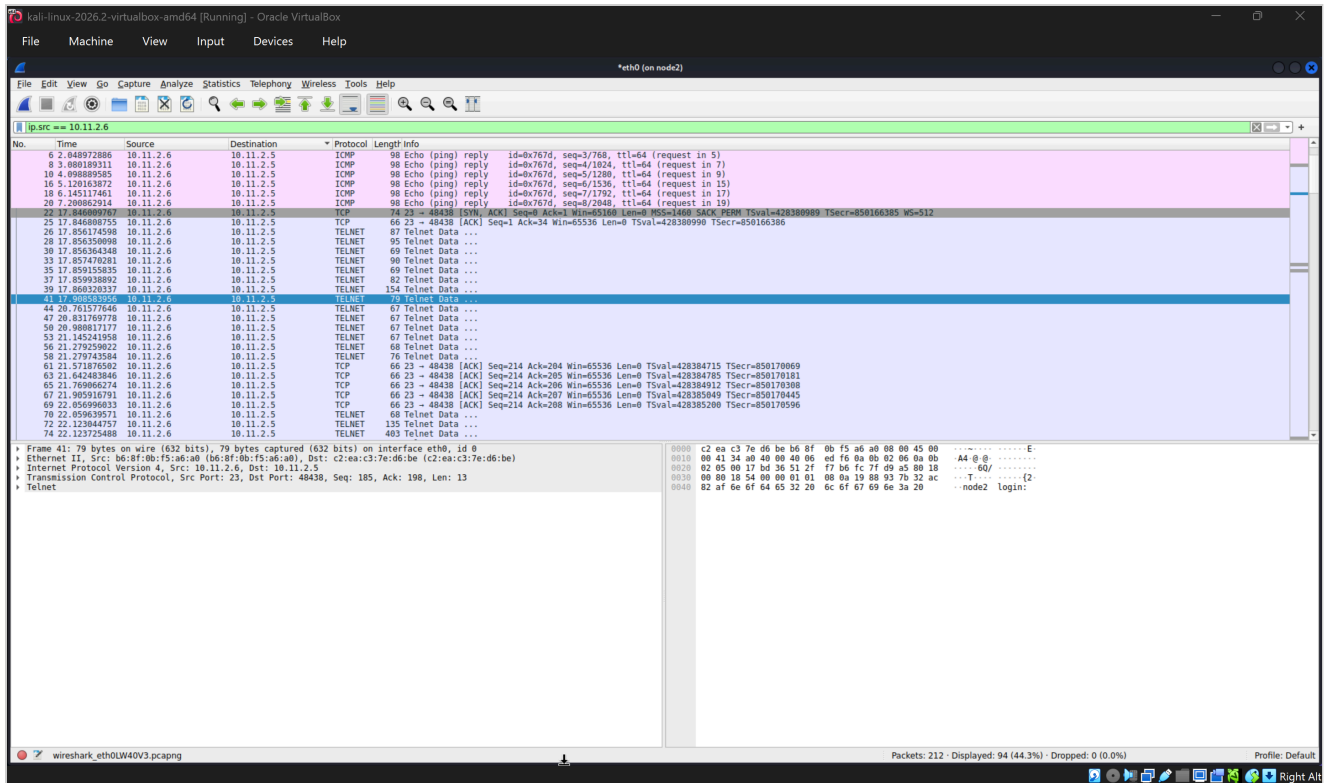


Figure 7. `ip.src == 10.11.2.6`: only node2's ICMP replies, TCP ACKs, and Telnet data.

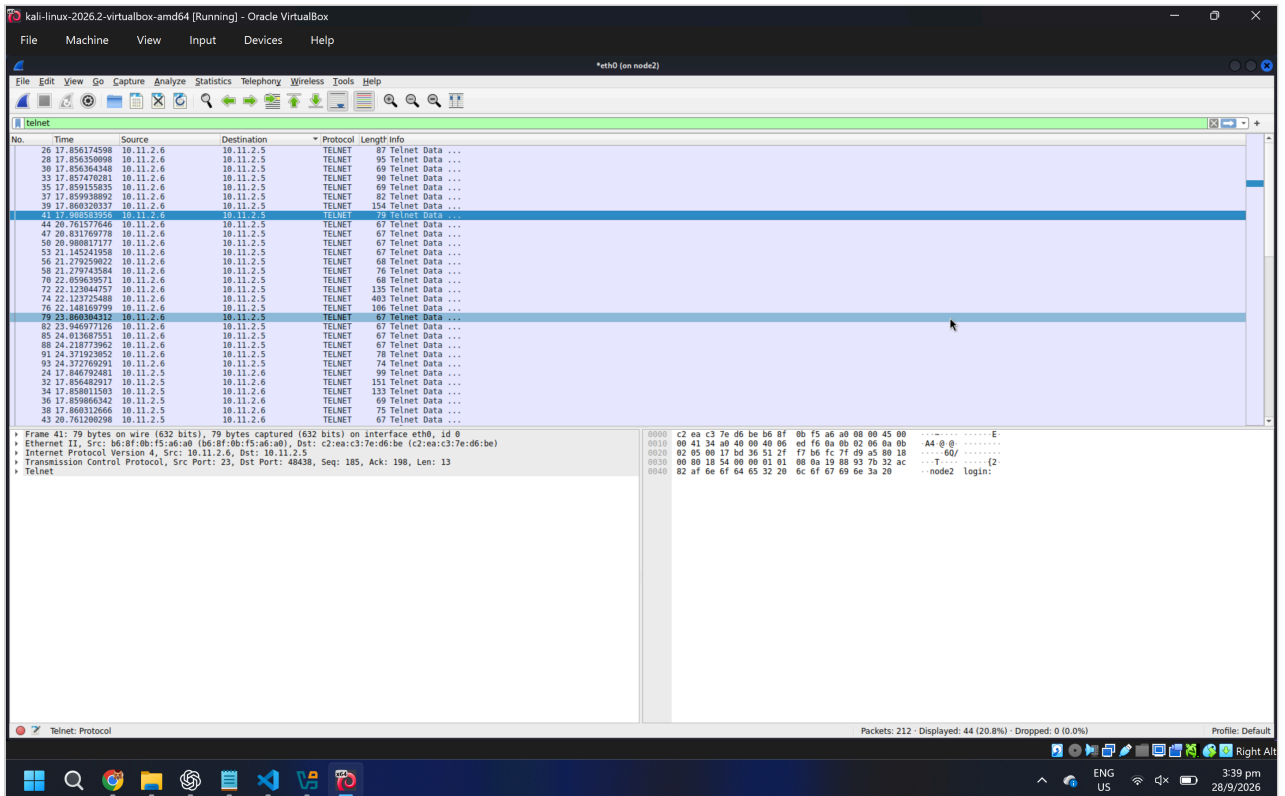


Figure 8. `telnet` filter: Telnet data between node1 and node2 on TCP port 23.

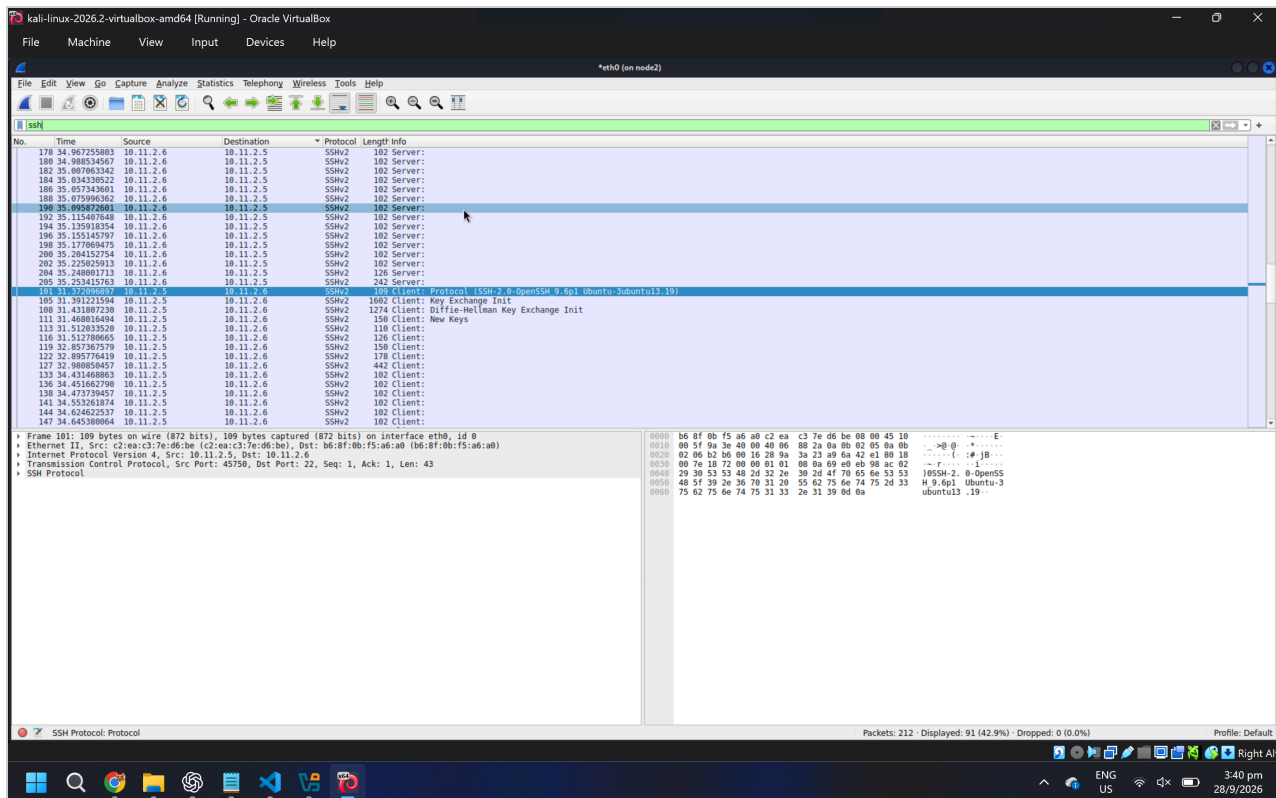


Figure 9. ssh filter: protocol banner, key exchange, and encrypted packets (Info shows only "Client:"/"Server:").

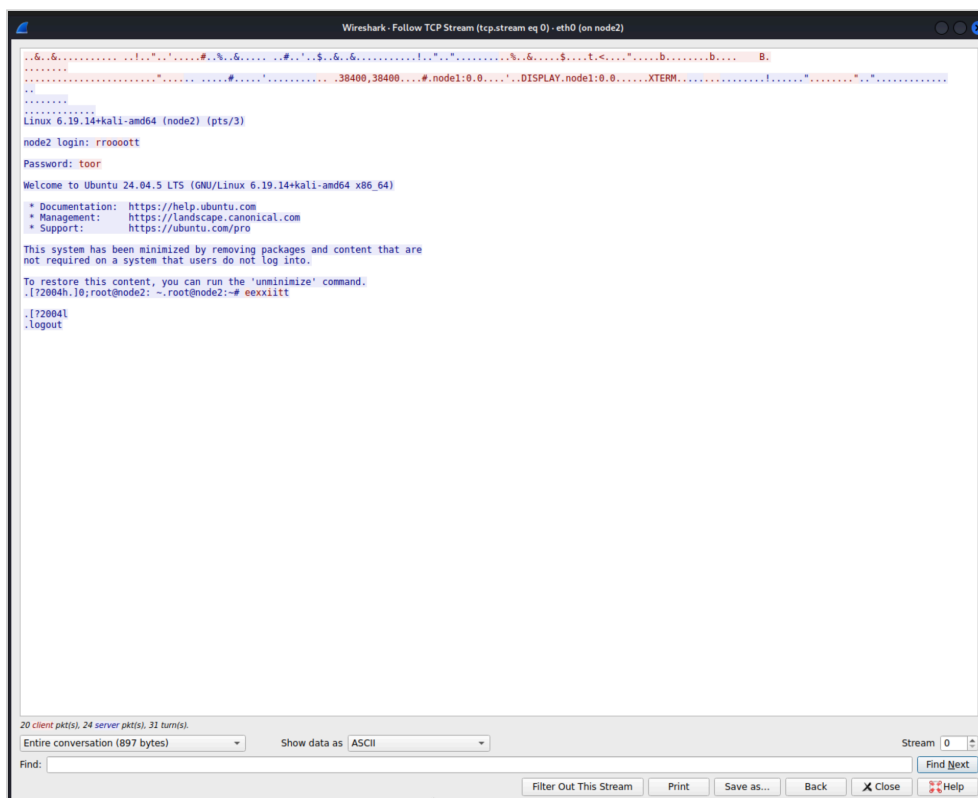


Figure 10. Follow TCP Stream of the Telnet session: username root and password toor in plaintext.

The username appears as rroooott because the client sends each character and the server echoes it back. The password toor appears once because the server does not echo it, but it is still sent in plaintext.

Q: Why is Telnet a greater risk than SSH?

Any passive sniffer on the path can read Telnet credentials, commands, and output (Figure 10), then reuse the credentials or hijack the session. SSH exposes only encrypted data, and host-key verification detects

server impersonation.

3. Source-IP Spoofing

`spoof_icmp.py` (run on node1) builds one ICMP Echo Request with a forged source address from the documentation range `198.51.100.0/24` (RFC 5737):

```
from scapy.all import ICMP, IP, send

packet = (IP(src="198.51.100.10", dst="10.11.2.6")
          / ICMP(type="echo-request", id=0x5002, seq=1)
          / b"Network Security Lab 1")
send(packet, iface="eth0", verbose=False)
```

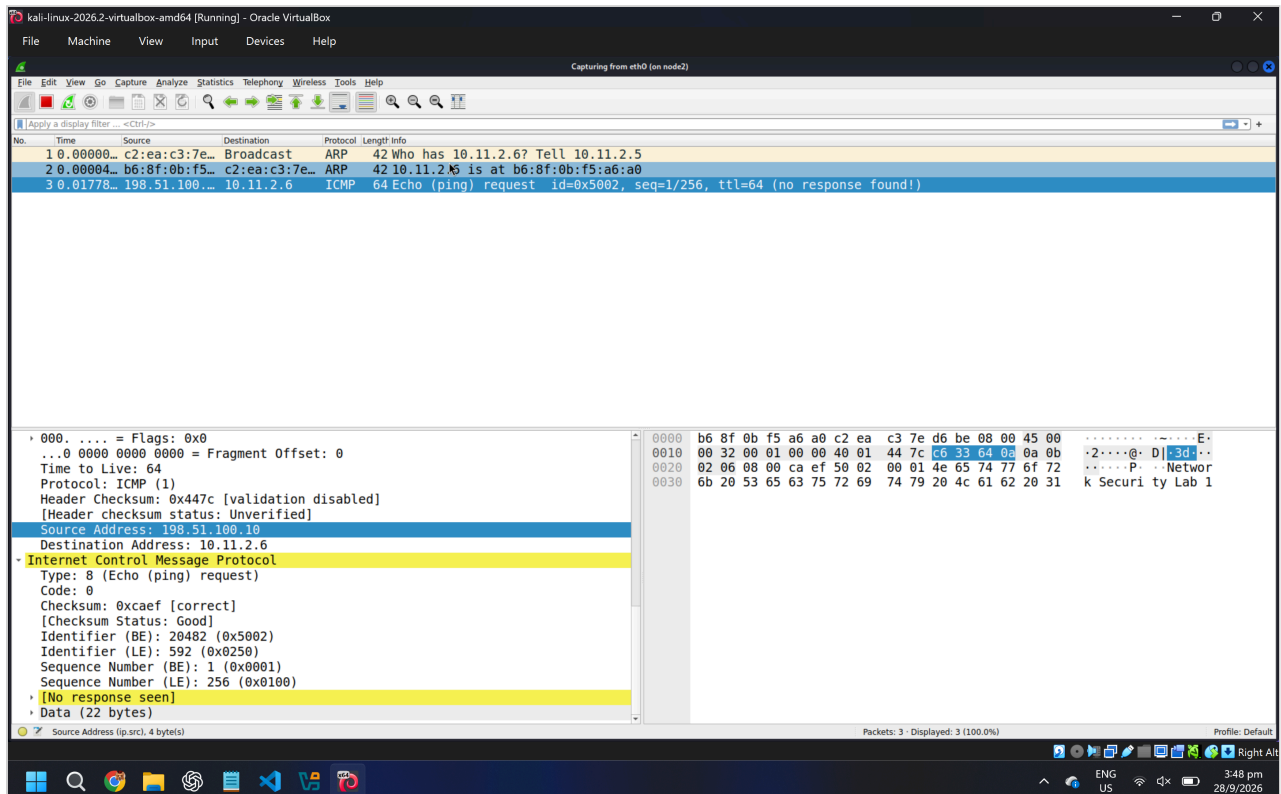


Figure 11. Wireshark on node2: Echo Request with source `198.51.100.10`, ID `0x5002`, seq 1, "no response found".

Q: What source IP does node2 observe? Is it node1's real address?

node2 sees `198.51.100.10`, not node1's real address `10.11.2.5`. The sender sets the source IP field freely and doesn't authenticate it, so the source IP of an individual packet is not proof of origin.

Q: Does node1 receive the reply? Where would it be sent?

No. node2 sends the Echo Reply to the packet's source address, `198.51.100.10`, because that is the only return address it has. That address is outside `10.11.2.0/24`, so node2 routes the reply via its default route. In this lab, that route is the `sink0` dummy interface, where the reply is discarded, so no reply appears on `eth0` (Figure 11). On a real network, the reply would go to whoever owns the forged address, so the spoofer never sees it.

Q: Compare normal and spoofed ICMP packets. What else indicates suspicious traffic?

Field / behaviour	Normal ping (node1)	Spoofed packet
Source IP	<code>10.11.2.5</code> (local subnet)	<code>198.51.100.10</code> (off-subnet)

Source MAC	c2:ea:c3:7e:d6:be (node1)	c2:ea:c3:7e:d6:be : node1's MAC, which reveals the real sender
TTL	64	64: no router hops, inconsistent with an off-subnet source
Frame / payload	98 bytes; standard 56-byte ping payload	64 bytes; custom 22-byte ASCII payload
ICMP ID / sequence	Per-process ID (e.g. 0x767d); sequence increases per request	0x5002 ; single request, seq 1
IP ID / flags	Kernel-assigned (62059, 62264, ...); DF set	1 (Scapy default); no DF
Reply	Echo Reply for every request (Figure 4)	None ("no response found")

The ARP request just before the spoofed packet (frame 1 in Figure 11) also came from 10.11.2.5 , not the claimed source. Useful indicators besides the source IP:

- An IP-to-MAC mismatch.
- An off-subnet source arriving directly on the local segment.
- A TTL that is inconsistent with the claimed origin.
- Requests with no replies.
- Non-standard payloads or header defaults.

4. AI-Based Traffic Analysis

Conclusion: An attack occurred. An unknown host, 10.0.2.8 (MAC 08:00:27:25:f3:ec), spoofed node1, the trusted workstation 10.0.2.6 (workstation01). It injected 800 forged messages into node2's (10.0.2.7) monitoring service: first a burst of compromise events, then a flood of false "all clear" reports.

4.1 Features extracted and why

analyze_incident.py (Scapy) exports all 4,061 packets to incident_packets.csv with these fields:

Feature	Relevance
Source and destination MAC	Identifies the device that actually transmitted each frame on the LAN
Source and destination IP, UDP ports	Claimed identity and service (10.0.2.6:40000 → 10.0.2.7:5514, syslog-style)
ARP sender IP/MAC	Shows which IP each MAC legitimately owns
IP TTL and IP ID	Fingerprints the sending network stack
Timestamp	Timeline reconstruction and burst detection
Payload fields: host, seq, severity, event, status, user	What each message claims; sequence numbers should be unique and increasing

4.2 Use of AI and analysis techniques

- **LLM-assisted coding:** GitHub Copilot generated the Scapy PCAP-to-CSV extractor and aggregation queries. I verified the output against total packet counts and raw payload samples.
- **LLM-assisted review:** The LLM read payload samples and summarized event distributions, flagging security-sensitive events that were absent from the baseline.
- **Rule-based anomaly detection:** Grouping by (source IP, source MAC) flagged one IP with two MACs. ARP data, sequence-number collisions, IP ID behaviour, and 30-second time bins then characterized the anomaly.

4.3 How the traffic supports the conclusion

Observation	Legitimate sender	Attacker
Claimed source IP	10.0.2.6	10.0.2.6
Source MAC	08:00:27:5a:87:bc	08:00:27:25:f3:ec
ARP identity of MAC	10.0.2.6 (17 ARP requests)	10.0.2.8 (2 ARP requests for node2)
Messages	3,200 across the entire capture	800, from 12:35:59 to 12:40:59
Sequence numbers	1–3200, all unique	602–2219, 713 unique; all reuse legitimate numbers, 698 with different content
IP ID	Increasing (43957, 43960, 44015, ...)	Random (39832, 55505, 8425, ...)
Event types	Heartbeat, CPU/memory, backup, auth, DNS, SSH running	Root/persistence events, then health and security "all clear"

The same claimed IP, ports, and host name (workstation01) are paired with a MAC that ARP ties to 10.0.2.8. Duplicate sequence numbers with conflicting content and a different IP ID pattern confirm a

second sender injecting into the stream. Throughout the attack, the real workstation continued sending routine telemetry and reported none of the attacker's events.

4.4 What the attacker did

1. **Discovery:** 10.0.2.8 resolved node2's MAC through ARP.
2. **Phase 1, about 12:36:00 (<30 s):** Forged ALERT/WARN messages claimed workstation01 activity as user=root: SUDO_ROOT SUCCESS (57), FIREWALL_POLICY DISABLED (48), SSH_CONFIG MODIFIED (46), AUTHORIZED_KEY ADDED (40), and AUDIT_CONFIG MODIFIED (31).
3. **Phase 2, about 12:40:00–12:41:00:** A flood of reassuring messages: SYSTEM_HEALTH OK (112), SECURITY_SCAN CLEAN (98), FIREWALL_POLICY ENABLED (75), INTEGRITY_CHECK PASS (71), BACKUP COMPLETED, and SSH_SERVICE RUNNING.

4.5 Likely objective

The likely objective was to deceive the SOC by poisoning node2's monitoring data while impersonating a trusted host. Phase 1 records root access, SSH-key persistence, and disabled firewall and audit controls under workstation01's identity. Phase 2 then floods the server with "all clear" reports. As a result, the latest recorded state shows the system and its security services operating normally, which is exactly the confirmation management wants. The reused sequence numbers leave node2 with contradictory logs, which explains the inconsistency the SOC noticed.

Limitation: A network capture proves the messages were forged. It cannot show whether the reported changes actually occurred. workstation01's authorized_keys, SSH configuration, firewall, and audit settings should be checked directly, and 10.0.2.8 should be isolated and investigated.

Mitigation: Authenticate telemetry (TLS with client certificates or signed messages), alert on IP-to-MAC changes and duplicate sequence numbers, and enable ARP inspection or port security on the network.

5. Submitted Files

- spoof_icmp.py: Task 3 spoofed ICMP sender.
- analyze_incident.py: Task 4 PCAP feature extractor and anomaly summary.
- incident_packets.csv: extracted packet features from lab1_incident.pcapng.

Reproduce the Task 4 analysis: `python analyze_incident.py lab1_incident.pcapng --output incident_packets.csv`